

Memòria Pràctica I

Coneixement, Raonament i Incertesa

Grup:

G08:30_2

Membres:

Marc Sala - 1747246

Àlex Pozo - 1744555

Sofia Atienza - 1708916

Índex

1.- Introducció.....	pàg. 2
2.- Solució proposada.....	pàg. 3
2.1.- Part C (Exercici 1).....	pàg. 3
2.2.- Part B (Exercici 2).....	pàg. 7
2.3.- Part A (Exercici 3).....	pàg. 8
3.- Problemes trobats durant el projecte.....	pàg. 9
4.- Resultats obtinguts.....	pàg. 10
5.- Desenvolupament individual.....	pàg. 11
6.- Conclusions individuals.....	pàg. 12

1.- Introducció

El problema plantejat consisteix a resoldre de manera automàtica un tauler de mots encreuats. Donat que aquest es pot modelar com un CSP (Constraint Satisfaction Problem), la resolució consisteix a trobar una assignació de paraules d'un diccionari predefinit a les caselles d'un tauler, complint amb les condicions pròpies del joc. S'analitza també, la seva capacitat d'escalabilitat segons la mida del diccionari.

La pràctica es divideix en 3 parts, de menor a major complexitat:

- ❖ **Part C (Exercici 1):** Aborda el problema mitjançant l'algorisme de Backtracking pur sobre un tauler i com a recurs, s'utilitza un conjunt de diccionaris de mida reduïda com a base de la cerca.
- ❖ **Part B (Exercici 2):** Optimitza el procés anterior integrant la variant de Forward Checking, reduint l'arbre de cerca mitjançant el filtratge anticipat dels dominis de les variables no assignades.
- ❖ **Part A (Exercici 3):** Posa a prova l'escalabilitat i l'eficiència del codi aplicant la solució de la part B a un escenari a gran escala amb un diccionari de més de 580.000 paraules.

Aquesta pràctica té com a objectiu principal assimilar els conceptes de cerca amb satisfacció de restriccions (CSP) i aplicar de forma correcta i funcional tan l'algorisme Backtracking (BT) com la seva variant amb Forward Checking (BT-FC). Es diferencien en com i quan es detecten conflictes amb les variables futures no assignades, aprofundirem més sobre aquest aspecte al llarg del desenvolupament de la memòria.

2.- Solució proposada

2.1.- Part C (Exercici 1):

El flux d'execució de la nostra solució, es divideix en tres fases principals:

- 1) Preprocessat de Dades: lectura de fitxers i dicP per mida
- 2) Identificació de Variables: detecció de forats buits al tauler
- 3) Cerca per Backtracking: exploració recursiva

1) Preprocessat del Diccionari i del Tauler

En lloc d'emmagatzemar les 107 paraules en una llista, les organitzem en un diccionari *dicP* on les claus són les longituds de les paraules (s'entra més en detall en l'apartat de estructures utilitzades). Això permet que el programa accedeixi directament a *dicP[N]*, reduint el temps de cerca. Es carrega també la matriu (*cross*): Es llegeix, convertint el tauler en una matriu bidimensional de caràcters.

2) Detecció Automàtica d'Espais / Variables (*dicC i espais*)

El programa analitza dinàmicament la matriu *cross*: Determina la longitud mínima de paraula existent al diccionari ($minim = \min(dicP.keys())$). Recorre totes les files i columnes comptant caselles buides ('0') consecutives i, si troba una seqüència de '0' d'amplada igual o superior al mínim de lletres de la paraula, crea una variable i la guarda a la llista *espais* amb una estructura de tupla:

- Per a espais horitzontals: ('H', fila, longitud, columna_inici)
- Per a espais verticals: ('V', fila_inici, longitud, columna)

3) Cerca Recursiva per Backtracking (*backtracking*)

L'algorisme *backtracking* (*espais*, *index*, *matriu*, *visitats*) recorre la llista d'*espais* (*index*) provant possibles candidats. Per a cada *espai* per tant, es filtren primer possibles candidats: S'obtenen les paraules de *dicP[llargada]* i es descarten les que ja s'han utilitzat anteriorment (*p not in visitats*).

Per tal de fer un *backtrack* sense haver de copiar tota la matriu del tauler, el nostre codi només guarda una còpia de la línia que ha canviat:

- Horitzontal: *copia* = *list(matriu[x])* (copia la fila).
- Vertical: *copia* = [*matriu[i][y]* for *i* in *range(len(matriu))*] (copia la columna).

Seguidament, es valida i es col·loca la paraula (*posarParaulaH*/*posarParaulaV*):

Es recorre la paraula caràcter a caràcter, només s'accepta si la casella conté un '0' (buit) o si ja conté la mateixa lletra. Si encaixa, la funció escriu directament la lletra a la matriu i retorna True. En el cas de que hi hagi qualsevol conflicte, s'atura i retorna False.

Finalment, es processa el Backtrack: Si la paraula és vàlida, s'afegeix a visitats i es fa la crida recursiva per al següent espai (index + 1). En el cas de que la crida recursiva posterior no trobi solució, es desfà el camí (*visitats.pop()*) i es restaura la fila/columna de la matriu utilitzant la còpia guardada).

Estructures de dades utilitzades:

- Estructura per emmagatzemar espai:

Inicialment, creem un diccionari que emmagatzema els espais disponibles en horitzontal i el mateix amb els verticals, del tauler. Ens servirà per després crear una llista *espais*=[], on cada posició d'aquesta conté informació sobre l'espai corresponent. Conté la direcció de l'espai ('H' si és horitzontal i 'V' si és vertical), la fila on comença, la longitud de l'espai i, finalment, la columna on comença.

```
dicC = {'Horitzontal': {}, 'Vertical': {}}
```

```
espais = []
#Tindrem 'H' = horitzontal i 'V' = vertical
direccio = espais[index][0]
x = espais[index][1] #Fila on comença l'espai
llargada = espais[index][2] #longitud de l'espai = nombre de lletres
y = espais[index][3] #Columna on comença l'espai
```

- Estructura per emmagatzemar el diccionari:

Emmagatzema un diccionari on la clau de cada posició, representarà un índex que indica la longitud de la paraula de manera que, des d'un principi, es filtren les paraules i es classifiquen segons la seva longitud. Això ens permet quan tinguem per exemple un forat de 5 caselles, accedir directament a la posició 5 del diccionari on estan totes les paraules possibles amb 5 lletres.

```
dicP = dict()
```

- Estructura per emmagatzemar la matriu d'encreuament:

És pròpiament el tauler, on actualitzem les posicions d'aquest, quan l'anem modificant i per tant, és el que carreguem al final per a visualitzar l'estat del joc.

```
cross = list()
```

Per a aquesta primera part disposem dels següents arxius:

- 1) Un fitxer anomenat *diccionari_CB_v3.txt* on hi ha 107 paraules diferents
- 2) Un document on hi ha l'esquema d'un tauler de mots encreuats *crossword_CB_v3.txt*

A continuació, s'especifiquen les característiques del problema:

- **Variables:**

En aquest cas, les variables són els diferents espais contigus (horitzontals i verticals) on s'ha d'ubicar una mot dins del tauler del joc.

En el codi, definim les variables segons cada element de la llista *tasques*:

La tupla ('H'/'V', *x*, *llargada*, *y*) conforme una variable.

- 'H' o 'V' indica l'orientació (horitzontal o vertical).

- x i y són les coordenades inicials de la primera casella.
- *llargada* és el nombre de lletres que caben en el forat.

Si el tauler conté N caselles, tindrem N variables: $X = \{X_1, X_2 \dots X_N\}$.

- **Domini de les variables:**

El domini D_i d'una variable X és el conjunt de paraules del diccionari que tenen exactament la mateixa longitud que el forat associat a X .

$D_i = \{p \text{ que pertanyen a Diccionari} \mid \text{len}(p) = L_i\}$

En el codi, l'ús de l'estructura *dicP* fa que el domini d'una variable de longitud L_i vingui donat directament per la llista *dicP*[L_i].

- **Restriccions del problema:**

El problema presenta 3 restriccions:

- Restricció global de No Repetició: En primer lloc, una paraula no es pot utilitzar més d'una vegada. En el codi això es comprova mitjançant la condició: *p not in visitats* (funció de *backtracking* i funció de *actualitzarDominis*).
- Restricció Binària de Creuament: Quan dues paraules en diferents posicions (una en vertical i l'altre en horitzontal) es creuen, el caràcter en la posició de creuament ha de ser idèntic per als dos mots. En el codi, es comprova a les funcions *posarParaulaH* i *posarParaulaV*: una casella només s'accepta si conté un '0' (està buida) o si ja conté exactament el mateix caràcter (*matriu*[x][y] == *char*).
- Restricció Unària (mida): la mida de la paraula assignada p ha de coincidir exactament amb la longitud del espai disponible.

- **Tamany de l'espai de solucions inicial:**

Inicialment el codi prefiltra les paraules segons la mida del forat abans de començar la cerca, l'espai de solucions inicial que l'arbre de cerca explora, ve donat pel producte de les mides dels dominis de cada variable:

$\text{Espai Total} = |D_i| (i=1 - N) = |D_1| \times |D_2| \times |D_3| \dots |D_N|$

Si no s'hagués filtrat inicialment per longitud o unicitat, l'espai total seria

$\text{Espai Total} = M^N = 107^N$, sent M el diccionari amb les 107 paraules.

- **Estratègies utilitzades per reduir la profunditat mitjana de les branques:**

- Filtratge previ per longitud (*dicP*):

En lloc de provar totes les 107 paraules del diccionari en cada iteració, es redueix el factor de ramificació filtrant el domini per la mida del forat (*dicP[llargada]*). Això evita explorar branques innecessàries que igualment anaven a fallar al fer la primera comprovació, la de longitud.

- Poda immediata (Pruning):

A les funcions *posarParaulaH* i *posarParaulaV*, en el moment en què un caràcter de la paraula no coincideix amb la lletra ja existent al tauler, la funció retorna False. Això fa que es talli la branca en aquell precís moment sense fer la crida recursiva.

En aquesta part (Part C) no s'ha aplicat encara cap estratègia més avançada com Forward Checking, ja que es demana a la següent part. És per això que l'ordre d'exploració de les variables ve donat per l'ordre de la llista *tasques*.

2.1.- Part B (Exercici 2):

Un cop hem acabat el codi base de la pràctica, ens hem dedicat a afegir tècniques i millores per optimitzar-lo i poder resoldre el crossword_A en el menor temps possible.

En aquest primer apartat, hem implementat la funció `actualitzarDominis()`, que aplica Forward Checking un cop s'ha afegit una paraula a la matriu. D'aquesta manera, la combinació dels dos algorismes integra l'exploració ordenada del Backtracking amb el filtratge preventiu del Forward Checking.

1. Quina és la diferència entre els dos mètodes?

A diferència del Backtracking, Forward Checking s'anticipa a les següents iteracions. Si detecta almenys un forat sense cap paraula que s'hi pugui assignar, poda aquesta branca perquè no existeix solució possible si la continuem explorant.

2. Com millora aquest respecte l'anterior?

Backtracking = Temps de resolució: 0.0007 segons

Backtracking + Forward Checking = Temps de resolució: 0.0007 segons

3. Heu canviat les variables, el domini o les restriccions del problema?

4. Has canviat l'estratègia utilitzada per reduir la profunditat mitjana de les branques utilitzada en l'apartat anterior?

2.1.- Part A (Exercici 3):

3.- Problemes trobats durant el projecte

4.- Resultats obtinguts

5.- Desenvolupament individual

(cada membre explica la seva contribució)

6.- Conclusions individuals

(cada membre extreu les seves pròpies)